Data Structures & Algorithms

Assignment 2: AVL Tree Deletion & Integrity Verification

Problem Statement

Implement an AVL tree data structure in C++ that supports insertion and deletion while maintaining strict balancing rules. The deletion algorithm must specifically handle nodes with two children by allowing the user to choose between using the inorder successor or the inorder predecessor as the replacement. The program must dynamically display a log of any structural rotations (Left, Right, LR, RL) triggered during the rebalancing phase. Finally, the program must verify the structural integrity of the tree before and after deletion by confirming that the inorder traversal remains strictly sorted and that all node balance factors and heights are mathematically valid.

Algorithm

1. **Initialize the Tree:** Start with an empty tree (root = NULL). Prompt the user for the number of nodes (N) and sequentially read N values.
2. **Insert and Balance:** For each value, perform a standard Binary Search Tree (BST) insertion. As the recursion unwinds, update the height of each node and calculate its balance factor. If the balance factor is outside the range [-1, 1], apply the appropriate rotation (Left, Right, Left-Right, or Right-Left) to rebalance the tree.
3. **Pre-Deletion Verification:** Perform an inorder traversal to print the tree. Verify that the elements are in strictly ascending order and that every node's calculated balance factor and stored height are correct. Display the validation results.
4. **Target Selection:** Prompt the user for the key to delete and their preferred replacement method (1 for Inorder Successor, 2 for Inorder Predecessor).
5. **Search and Delete:** Traverse the tree to find the target key.
    - *Case A (0 or 1 child):* Temporarily store the child (if any), delete the current node, and return the child to bypass the deleted node.
    - *Case B (2 children, Successor method):* Find the node with the minimum value in the right subtree. Copy its key to the target node, then recursively delete that minimum node.
    - *Case C (2 children, Predecessor method):* Find the node with the maximum value in the left subtree. Copy its key to the target node, then recursively delete that maximum node.
6. **Rebalance Upward:** As the deletion recursion returns, update the heights of the current nodes. Calculate their balance factors. If an imbalance is detected, identify the specific imbalance case, perform the required rotation(s), and print the rotation action to the console.
7. **Post-Deletion Verification:** Re-run the inorder traversal and integrity checks to guarantee the tree remains perfectly sorted and balanced after the deletion and rotations.
8. **Cleanup:** Recursively free all dynamically allocated memory for the tree nodes and terminate the program.

System Flowchart

The following flow diagram outlines the logic and execution path for the AVL Tree insertion, balancing, verification, and deletion routines.

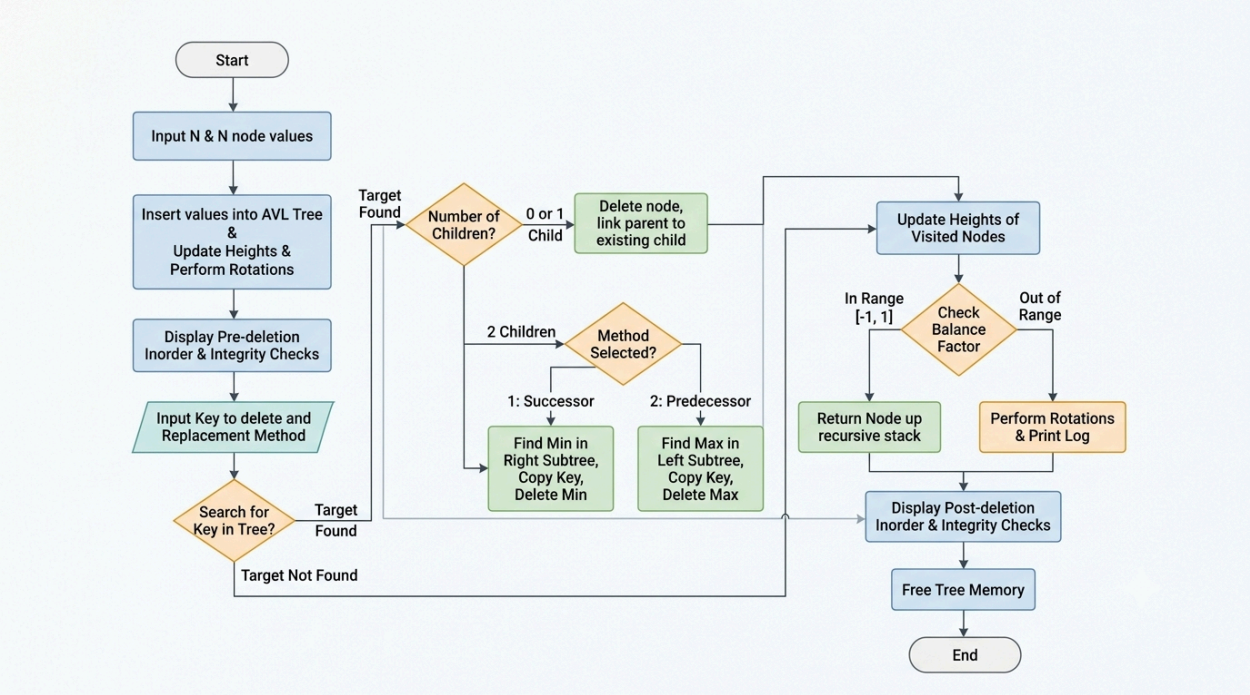


Figure 1: Complete execution flowchart for AVL Tree node deletion and dynamic rebalancing.